Secure Web
Link Analyzer



A Hybrid Rule-Based and Machine-Learning Approach for Pre-Visit URL Risk Assessment

PROJECT GUIDED BY

Mrs. V.P. Anitha

PRESENTED BY

Seenivasan E (23619134) · Harish Sagar S (23619115)

Abstract :

The Secure Web Link Analyzer is a cybersecurity solution designed to identify and evaluate potentially malicious or unsafe web links before users access them. As phishing attacks, malware distribution, and fraudulent websites continue to increase, the system provides an automated mechanism to analyze URLs using multiple security parameters, including domain reputation, HTTPS validation, blacklist verification, URL structure analysis, and machine learning-based threat detection. The analyzer classifies links as safe, suspicious, or malicious and presents users with a comprehensive security assessment along with risk explanations. By integrating real-time threat intelligence and intelligent detection techniques, the system helps prevent phishing attacks, credential theft, malware infections, and unauthorized data access. Its user-friendly interface enables individuals and organizations to verify links quickly, enhancing cybersecurity awareness and reducing the risk of online attacks. The Secure Web Link Analyzer offers an efficient, scalable, and proactive approach to web security, contributing to safer internet browsing and improved protection against evolving cyber threats.

Introduction :

- Every day, people click links shared in emails, messages, and social media without knowing if they're safe.
- Phishing links disguise themselves as trusted sites — like banks or PayPal — to steal passwords and money.
- Traditional blocklists only catch a scam link after many people have already been tricked by it.
- Secure Web Link Analyzer checks any link instantly using AI and rule-based checks — before it's ever opened.
- It gives a clear verdict — Safe, Suspicious, or Malicious — along with plain-English reasons anyone can understand.



Problem Statement :

No simple pre-check

Users lack a simple mechanism to verify unfamiliar URLs before visiting them.

Convincing imitation

Phishing and malicious websites closely imitate legitimate domains and pages.

Blocklists lag behind

Blocklist-only approaches miss newly generated or previously unseen malicious URLs.

Opaque verdicts

Many detection systems provide limited explanations for end users.

Need: a pre-visit analyzer that combines multiple security indicators with an understandable risk assessment.

Objectives :

01

DETECT

Identify potentially phishing and malicious URLs before access.

02

ANALYZE

Evaluate URL structure, HTTPS, domain and threat indicators.

03

CLASSIFY

Categorize links as Safe, Suspicious or Malicious.

04

EXPLAIN

Provide a risk score and reasons behind the classification.

Literature Survey :

Title	Author	Methodology	Addresses	Limitation
Machine learning based phishing detection from URLs (2024)	Sahingoz, Buber, Demir & Diri	URL-only lexical features, Random Forest & other ML classifiers	Real-time phishing detection without contacting third-party services	Outputs only a numeric score — no plain-English explanation
Detecting Phishing Domains Using Machine Learning (2023)	Alnemari & Alshammari	Comparison of four ML models (Applied Sciences)	Accuracy comparison across classifiers for phishing domains	Focuses on model comparison, not an explainable rule layer
Look before you leap: Detecting phishing web pages by exploiting raw URL and HTML characteristics (2024)	Opara, Chen & Wei	Combines raw URL + HTML page features	Improves accuracy by pairing URL and page-content signals	Must fetch and parse the live page — can't judge a link without visiting it
Phishing URL detection with neural networks: an empirical study (2024)	Ghalechyan, Israyelyan, Arakelyan et al.	Neural network trained on open-source + private datasets	Production-grade phishing URL detection	No interpretable, reason-based verdict for end users
Explainable Machine Learning for Phishing Feature Detection (2024)	Calzarossa, Giudici & Zieni	Ensemble tree-based ML model paired with a feature-importance explainability layer	Accurate phishing predictions plus explanations of which features drove the decision	Explanations are feature-importance scores, not a plain-English verdict end users can read directly

Research Gap :

EXISTING ML SYSTEMS

Strong prediction, but often only a score or classification.

URL + HTML SYSTEMS

Can improve accuracy but may require fetching the page.

END-USER NEED

Fast pre-visit decision with clear, understandable reasons.

MULTI-PARAMETER ANALYSIS + EXPLAINABLE RISK SCORING

Proposed Methodology :



URL Input

User enters or pastes a URL.

Feature Extraction

Extract lexical and security-related URL features.

Security Checks

HTTPS, domain validity, blacklist / reputation and suspicious patterns.

Risk Analysis

Combine rule-based indicators and ML prediction.

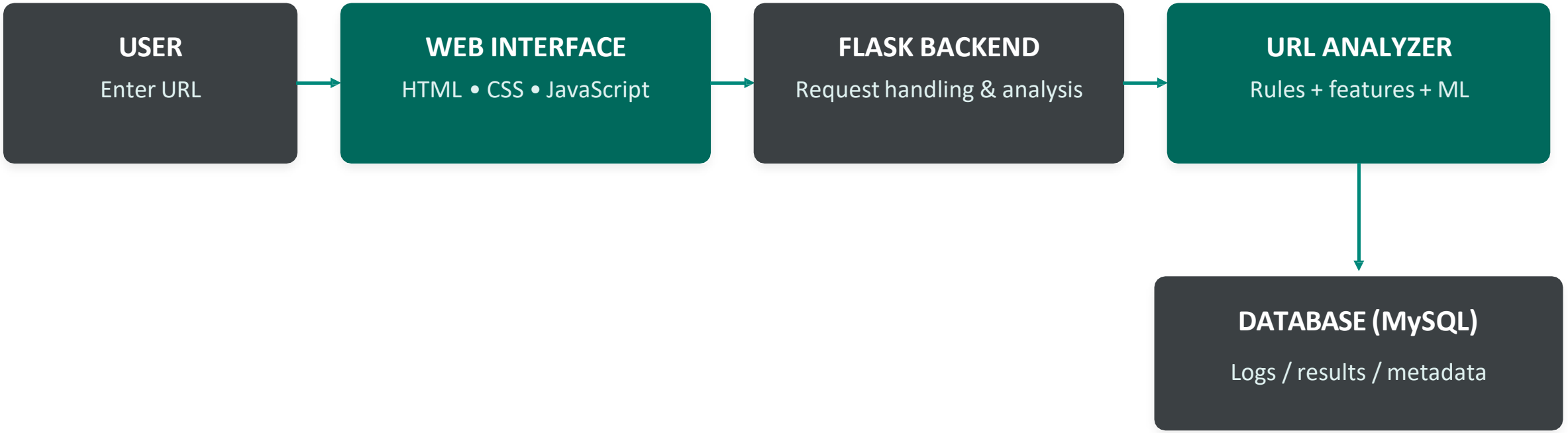
Classification

Generate Safe / Suspicious / Malicious verdict.

Explanation

Display risk score and contributing reasons.

System Architecture :



Hardware & Software Requirements :

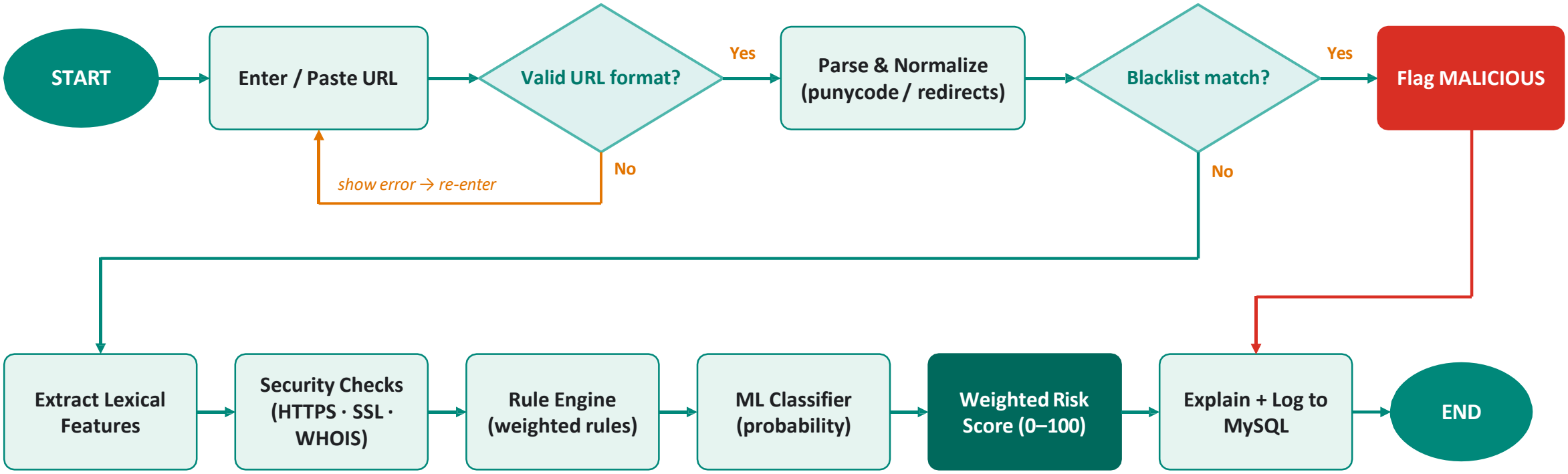
HARDWARE

- Intel Core i3 or above
- 4 GB RAM minimum; 8 GB recommended
- 20 GB free storage
- Network connection

SOFTWARE

- Windows / Linux
- HTML, CSS, JavaScript
- Python + Flask
- MySQL
- scikit-learn, pandas (ML module)
- VS Code / PyCharm

System Design — Flowchart :



Verdict thresholds (configurable):

0 – 30 SAFE

31 – 60 SUSPICIOUS

61 – 100 MALICIOUS

Complete Project Implementation :

End-to-end integration of the web interface, Flask backend, analysis engine, ML model and storage layer.

01 Frontend UI

HTML, CSS and JavaScript URL input, result card and scan-history interface.

02 Flask Backend

POST /analyze request handling, URL validation, controller logic and API response.

03 Feature Extraction

URL length, special characters, IP host, subdomains, entropy and suspicious keywords.

04 Security + Rules

HTTPS/SSL, domain checks, typosquatting, blacklist cache and weighted rule scoring.

05 Machine Learning

Feature vector passed to the trained classifier (.pkl) to produce a prediction / probability.

06 Result + Storage

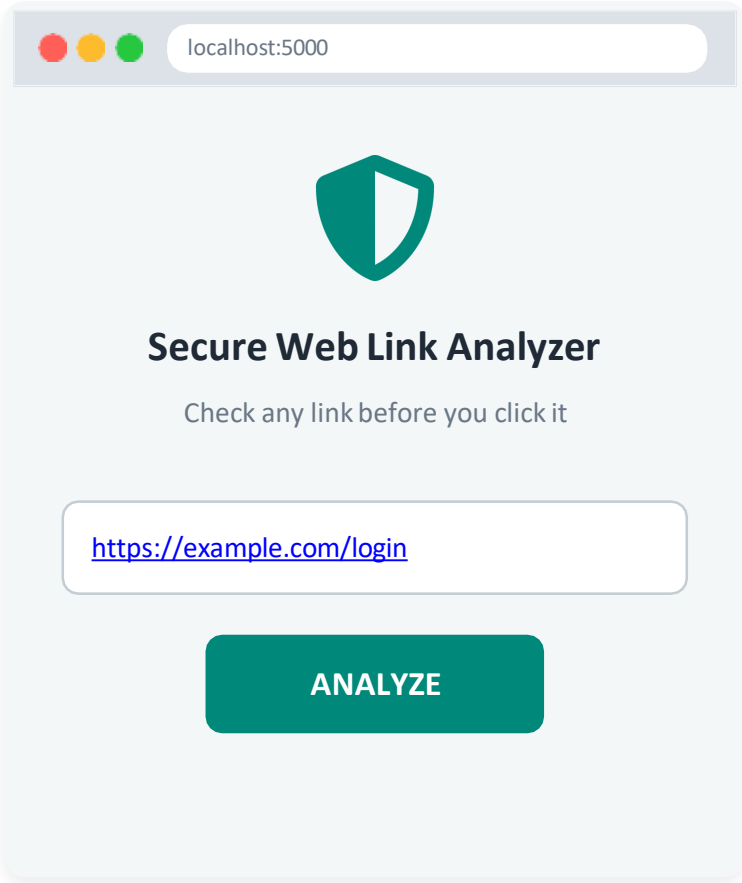
Combined risk score, Safe / Suspicious / Malicious verdict, explanation and MySQL logging.

FINAL INTEGRATED FLOW

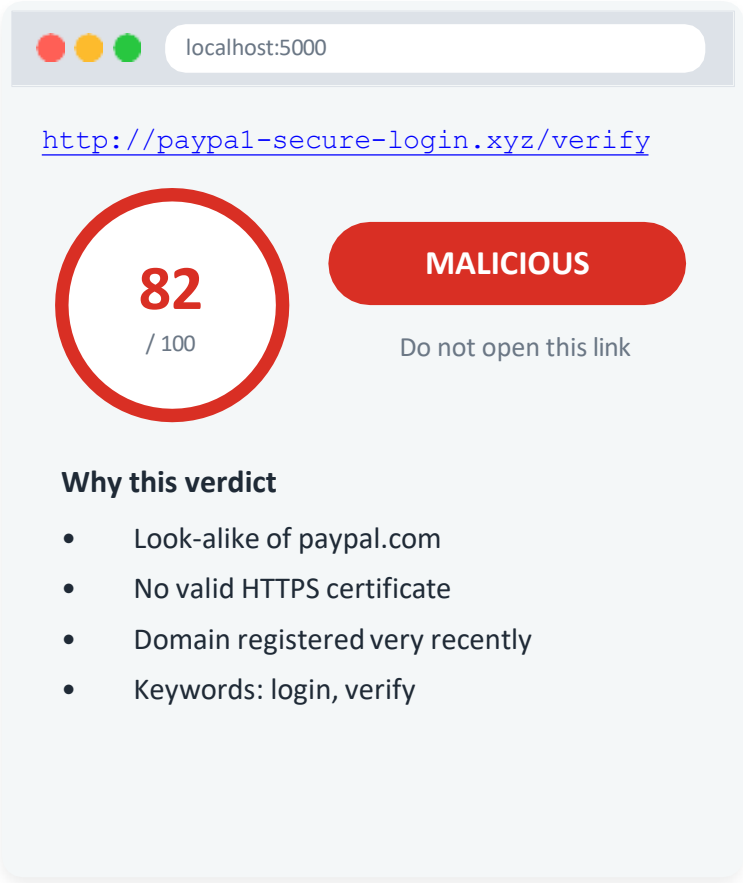
URL Input → Validate → Extract Features → Rules + ML → Risk Score → Verdict + Reasons → Log Result

Implementation overview is aligned with the architecture and module descriptions already present in the review deck.

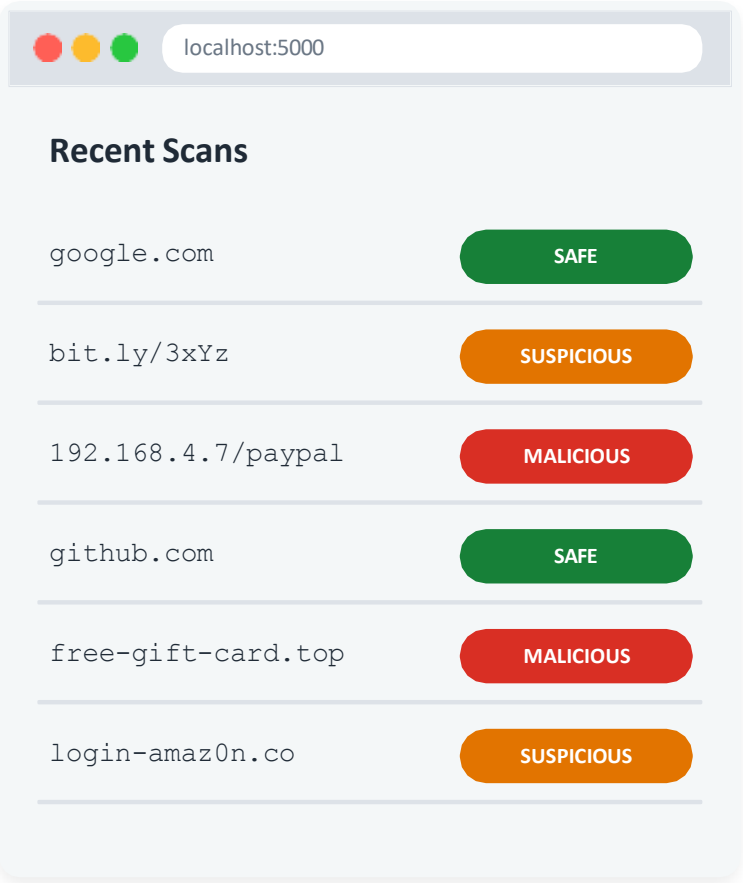
Prototype — UI Screens :



① Home — URL Input



② Analysis Result



③ Scan History

Working Demonstration :

POST /analyze → JSON response

```
{
  "url": "http://paypal-secure-login.xyz/verify",
  "risk_score": 82,
  "verdict": "MALICIOUS",
  "checks": {
    "https": false,  "domain_age_days": 12,
    "blacklisted": false, "typosquat": "paypal.com"
  },
  "reasons": [
    "Domain imitates paypal.com",
    "No valid HTTPS certificate",
    "Domain registered 12 days ago",
    "Suspicious keywords: login, verify"
  ]
}
```

Illustrative response format

LIVE DEMO FLOW

1. **Run the app**
   `python app.py → localhost:5000`
2. **Paste a URL**
   Enter the link to be checked
3. **Analyze**
   Features, checks, rules and ML run
4. **Read the verdict**
   Score, label and plain-English reasons
5. **Verify the log**
   Scan stored in MySQL history

DEMO TEST LINKS

SAFE	SUSPICIOUS	MALICIOUS
google.com	bit.ly link	IP-host login

Final Results & Performance Analysis :

Current measurable outcomes from the testing section, with ML metrics kept explicit where the review deck has not reported values.

8	6	2	75%
Test cases designed	Passed	In progress	Current pass rate

FUNCTIONAL TEST OUTCOMES

Scenario	Observed outcome
Legitimate HTTPS site	Safe - Pass
IP host / typosquatted brand	Malicious - Pass
'@' symbol / very long URL	Suspicious - Pass
Malformed input / URL shortener	Handled as expected - Pass
New domain / punycode homograph	Still in progress

ML PERFORMANCE METRICS

Accuracy	Not reported yet
Precision	Not reported yet
Recall	Not reported yet
F1-score	Not reported yet

PERFORMANCE ANALYSIS

The six completed functional cases show that the rule-based pipeline correctly handles common URL-structure risks and invalid input. The two remaining cases depend on more advanced domain-age and Unicode/homograph handling. Final ML accuracy, precision, recall and F1 should be inserted only after evaluating the trained model on a held-out test set.

Challenges Faced :

01 Slow WHOIS lookups

Challenge: Queries were slow and rate-limited.

Fix: Cached results with a timeout fallback.

02 Imbalanced dataset

Challenge: Far more benign URLs than phishing ones.

Fix: Class weights and resampling during training.

03 Shortened URLs

Challenge: bit.ly-style links hide the real target.

Fix: Resolve redirects before scoring.

04 API usage limits

Challenge: Threat-feed APIs cap free requests.

Fix: Local blacklist cache, refreshed on schedule.

05 Explaining ML output

Challenge: Model gives a probability, not reasons.

Fix: Map top features to plain-English templates.

06 Homograph domains

Challenge: Unicode look-alikes pass simple checks.

Fix: IDNA decoding + brand similarity check.

References :

01 Reyes-Dorta, N., Caballero-Gil, P., & Rosa-Remedios, C. (2024). Detection of malicious URLs using machine learning. Wireless Networks, 30, 7543–7560. https://doi.org/10.1007/s11276-024-03700-w

02 ENISA. (2023). ENISA Threat Landscape 2023. https://www.enisa.europa.eu/publications/enisa-threat-landscape-2023

03 Sahoo, D., Liu, C., & Hoi, S. C. (2017). Malicious URL detection using machine learning: A survey. arXiv:1701.07179.

04 Patil, D. R., & Patil, J. B. (2018). Malicious URLs detection using decision tree classifiers and majority voting technique. Cybernetics and Information Technologies, 18(1), 11–29.

05 Fortinet. (2023). What is URL phishing? https://www.fortinet.com/resources/cyberglossary/url-phishing

06 PhishTank — community phishing URL database. https://phishtank.org

07 Google Safe Browsing API documentation. https://developers.google.com/safe-browsing



Thank You